

Minecraft 建筑生成多智能体系统：从中文需求到可执行数据包

龙想 2300011196；石宇宸 2300011051

《大语言模型与信息决策》课程项目

2026 年 6 月

摘要

Minecraft 建筑生成任务同时涉及自然语言理解、空间规划、材料选择、几何约束、连通性校验和游戏数据包导出。若直接要求大语言模型输出具体方块坐标，系统容易出现非法材料、坐标不一致、房间不连通和命令不可执行等问题。本文介绍一套面向 Minecraft Java 的建筑生成多智能体系统。系统将大语言模型限制在高层语义决策层，使其输出风格、体块、材料、房间拓扑和设计意图等 JSON；随后由本地确定性几何引擎完成 CSG 体块生成、BSP 房间划分、A* 连通路径、室内装饰、质量校验、自动修复和数据包导出。项目从最初的单一建筑智能体逐步迭代为多智能体流水线，并进一步加入模板语料学习、设计法则蒸馏、审美审计与候选择优机制。当前版本支持无 API key 的 mock 模式，可生成 blueprint.json、architect_datapack、preview.html 和 run_report.md；全量测试结果为 176 项通过、0 项失败。

关键词：大语言模型智能体；Minecraft；多智能体系统；CSG；BSP；A*；数据包

1 引言

Minecraft 的开放世界为生成式建筑提供了直观的实验场景。用户可以用一句中文描述建筑需求，例如“建一个欧式大房子”或“建造一个木屋别墅”，但系统真正需要解决的问题远不止文本生成。一个可用的建筑结果必须包含合理尺度、入口、房间、楼梯、屋顶、窗、材料、内饰、路径和最终可执行的 Minecraft 命令。

项目早期的设想是让智能体直接把需求变成建筑蓝图或命令。实际尝试后，我们发现这一路径不稳定：大模型擅长描述建筑意图，却不擅长长期保持三维坐标、材料注册名和拓扑约束的一致性。由此，项目的核心问题被重新定义为：如何让大模型负责语义与设计决策，同时让本地程序负责确定性几何落地和工程校验。

本文围绕这一修改后的问题定义展开。系统目标不是制作一个实时连服机器人，而是构建从中文需求到 Minecraft 可安装数据包的完整、可复现、可测试流水线。

2 系统总体设计

系统采用“LLM 语义智能体与本地确定性几何引擎”相结合的混合架构。ArchitectAgent、Planner Agent、CreativeDesignAgent 等模块只输出高层 JSON，描述风格、材料、体块、房间拓扑、立面节奏和场地语义。CSGBuilder、BSPPartitioner、AStarPathfinder 等本地模块再把语义描述转换为合法方块网格。Decorator、QA、Repair 和 Optimizer 模块负责室内落块、约束修复、质量检查与命令压缩。

这种边界划分保留了大语言模型在开放需求理解和风格构思上的优势，也避免让模型直接承担坐标计算、Minecraft 材料校验和命令体积控制等确定性任务。系统还保留 mock

模式，当外部模型不可用时仍可生成完整输出，便于课程评阅和复现实验。

表 1 系统层次与模块职责

层次	代表模块	主要职责
语义层	ArchitectAgent, PlannerAgent	将中文需求转化为风格、材料、体块和房间拓扑 JSON
知识层	TemplateKnowledgeAgent	从本地 schematic 案例中检索可迁移设计语法
几何层	CSGBuilder, BSPPartitioner, AStarPathfinder	生成外壳、房间、门洞、楼梯和可达路径
质量层	QA, Repair, Optimizer	校验材料、连通性、模板法则覆盖和命令数量
导出层	Exporter	输出 blueprint.json、mcfunction 和可安装 datapack

3 方法

3.1 语义智能体分工

主流程位于 construction_method_v1。用户输入首先被解析为 buildSpec，随后 ArchitectAgent 生成外壳语义，MaterialPaletteAgent 基于 Minecraft Java 1.21.1 方块目录校验材料，PlannerAgent 生成房间节点与连通边，CreativeDesignAgent 决定体块变体、平面排序、屋顶、立面和场地策略。

后续智能体继续细化建筑表现。StructureAgent 给出支撑和荷载路径语义，FacadeAgent、RoofAgent、SiteLandscapeAgent 分别规划立面、屋顶和场地，InteriorDetailAgent 与 DecoratorAgent 负责房间级家具、灯光、植物和功能台面。整个过程中，LLM 的输出被约束为 JSON 语义对象，而不是直接写入坐标命令。

3.2 确定性几何落地

几何层是系统稳定性的关键。CSGBuilder 将主体、侧翼、门廊、塔楼等语义体块合并为稀疏 voxel 网格，并抽取外表面、楼板、屋顶和开窗。BSPPartitioner 根据楼层功能和房间权重划分室内空间；AStarPathfinder 根据拓扑关系生成门洞、楼梯和可通行路径。这样可以避免 LLM 直接坐标生成中的穿墙、断路和房间重叠问题。

导出层将最终网格转换为 Minecraft 数据包。玩家复制 architect_datapack 到世界 datapacks 目录后，执行 /reload 和 /function architect:run 即可触发 clear + build。系统同时输出 preview.html 和 run_report.md，便于不进入游戏时检查结构、流程和统计信息。

3.3 模板语料学习

项目后期加入 mc_templates 下的 64 个本地 schematic 模板。我们没有把模板用作方块级复制库，而是将其分析为可迁移的设计语法，包括场地入口、前景花园、体块轮廓、屋顶语言、立面深度和室内场景。生成时，TemplateKnowledgeAgent 检索相关案例，CreativeDesignAgent 和 Site、Roof、Facade、Interior 模块再使用这些语法。

为降低抄袭风险，系统记录 source_fusion_policy，要求多来源融合并控制单一来源占比。模板法则覆盖、审美审计和自动修复机制构成了后期质量闭环，使建筑不只满足“能生成”，还尝试拥有可解释的风格来源和设计一致性。

4 实现过程与构思迭代

项目的主要工作并不是一次性写出最终架构，而是在多轮实现中逐步修改问题定义。2026 年 5 月 28 日的初始版本建立了 Minecraft architect agent，并加入自动安装世界与建造模式；6 月 2 日，蓝图生成被拆分为多个子智能体；6 月 9 日，项目加入语义规划层和多智能体建筑技能；6 月 17 日前后，construction_method_v1 基本成型；6 月 18 日至 19 日，模板语料、schematic 分析、设计法则、审美审计、自动修复和候选择优陆续接入主流程。

这些变化反映了项目构思的转向。最初我们关注“如何生成一个房子”，后来逐步意识到课程项目更需要展示智能体系统如何分工、如何约束大模型、如何把开放语义落地为可靠工程产物。因此，系统从 prompt 驱动的单点生成，转向多 Agent 分工、语义 JSON、本地几何、模板知识和质量闭环。

5 实验与结果

当前全量测试为 176 项通过、0 项失败，其中包括课程提交文档测试；原项目功能基线为 173 项通过、0 项失败。测试覆盖 Architect fallback、Planner、CSG、BSP、A*、Decorator、模板法则、候选择优、LLM provider、材料目录和可视化输出等模块。

本地样例 out/2026-06-19-145532212 使用 prompt“建造一个木屋别墅”。run_report.md 记录了 LLM 调用状态、seed、几何统计、装饰数量、模板法则覆盖、QA 状态和 Minecraft 使用步骤。该样例说明系统并非只生成文本，而是在输出前经过结构、材料、连通和命令层面的检查。

表 2 本地样例运行结果

指标	结果	说明
建筑尺寸	43 x 16 x 57	来自 run_report.md 的几何校验结果
房间可达性	14/14	入口可达房间全部通过
QA 检查	9/9	结构、材料、入口和装饰检查通过
装饰数量	819	由室内与风格装饰模块写入
命令压缩	2692 -> 1251	压缩倍率约 2.15x，降低数据包函数体积
模板法则覆盖	100%	TemplateLawAutoRepairAgent 补强后达到 law-perfect

6 讨论

从课程要求看，项目的整体性体现在所有模块都围绕同一条主线协作：中文需求输入、智能体语义决策、本地几何落地、质量检查和数据包导出。项目的创新性主要体现在三点。第一，系统清晰区分 LLM 与程序算法的职责，避免让大模型直接处理坐标和命令。第二，模板语料被抽象为设计语法和审美法则，而不是简单复制。第三，项目保留 mock 兜底和自动化测试，使没有 token 或现场网络时仍能复现完整流程。

项目也存在局限。当前版本不是 Mineflayer 实时机器人，也不模拟玩家逐块放置；GDMC HTTP 客户端虽然保留，但主交付仍是数据包导出。自动审美评分可以发现法则覆盖问题，却不能完全替代玩家视角下的尺度、镜头、路径和 block readability 检查。后续可补充真实游戏截图、人工评分样本，以及更严格的视觉相似性和非复制检测。

7 结论

Minecraft Constructing Agents 构建了一条从中文建筑需求到 Minecraft 可执行数据包的完整流水线。项目的核心经验是：在开放生成任务中，大语言模型更适合作为语义和设计决策者，而不是直接坐标生成器；本地几何算法、材料目录、连通性校验和命令优化可以为其提供稳定落地能力。

通过多智能体分工、CSG/BSP/A* 几何引擎、模板语料学习和质量闭环，项目将一个模糊的建房需求转化为可复现、可检查、可安装的工程产物。相比最终单次生成效果，项目更重要的贡献在于展示了一个课程级智能体系统如何从想法、失败尝试和架构调整中逐步形成。

AI 辅助说明

本项目合理使用 AI 工具辅助开发和文档整理。AI 辅助主要用于代码修改建议、文档初稿整理、报告润色、测试清单和网页排版检查；项目方向、系统边界、架构取舍、运行调试、测试执行、结果确认和最终整合由小组完成。报告和网站不伪造运行截图；若最终版本需要图像材料，应使用 preview.html 或 Minecraft 游戏内真实运行结果截图。

参考资料

- [1] Minecraft-Constructing-Agents 项目仓库与 README, <https://github.com/CityC196/Minecraft-Constructing-Agents>。
- [2] 本地样例运行报告：out/2026-06-19-145532212/run_report.md。
- [3] 模板吸收与设计法则相关文档：docs/template-assimilation-plan.md。
- [4] 课程提交展示页与提交清单：docs/index.html, SUBMISSION.md。